

An Efficient Generative Intelligent Multiobjective Grasping Model for Kitchen Waste Sorting

Songyun Deng^{ID}, Ruiyuan Pei^{ID}, Li Zhou^{ID}, Hai Qin^{ID}, Wei Sun^{ID}, and Qiaokang Liang^{ID}, *Senior Member, IEEE*

Abstract—The key challenge in automated kitchen waste sorting on conveyor belts is how robots execute grasping tasks. Current methods, dependent on object detection, struggle to adopt generative architectures due to data scarcity and algorithmic inefficiency. To solve these issues, this article proposes solutions in three aspects: dataset construction, annotation data processing, and generative model lightweighting. First, we built the KWG2024 dataset, which includes 3116 RGB-D images, 18 231 standard annotations, and three new evaluation metrics. Second, we proposed two novel annotation processing methods to address the issue of excessive and incorrect grasps generated by generative networks in single-attempt grasp planning of multiobjects. Third, we constructed an efficient network with classification capabilities, AMI-LiteNet, by integrating the advantages of PP-LiteSeg and AMI-NanoNet. On the public Cornell dataset, AMI-LiteNet achieved a grasping accuracy of 98.3% and an efficiency of 118 frames per second (FPS). In extensive comparative experiments on KWG2024, our model reached a leading computational efficiency compared with recent state-of-the-art methods. It acquired a grasp planning generation accuracy of 87.96% while only requiring a computational effort of 2.79 GFLOPs. Finally, our model was deployed on an edge computing device and achieved a maximum grasping accuracy of 90.91% and a classification accuracy (CA) of 97.48% in our actual robotic system. To our knowledge, this is the first time that a generative grasp method has been applied independently to kitchen waste sorting tasks on a conveyor belt.

Index Terms—Embedded AI, generative network, kitchen waste sorting, multiobject grasping, RGB-D dataset.

I. INTRODUCTION

WITH the development of human society and the progress of science and technology, the automatic recycling of waste has become a global challenge. How to accurately and quickly identify and sort waste is the key issue in waste automatic recycling. In the recycling of kitchen

waste, the demand for automated sorting based on artificial intelligence and robotics is increasing due to the complexity of the recycling environment, the biological risk of manual sorting, and the persistent shortage of human resources [1]. Besides, traditional methods of kitchen waste disposal include landfill, incineration, and resource recovery [2]. However, nondegradable wastes such as metal and plastic products are often mixed with kitchen waste, posing a great challenge to the harmless treatment of kitchen waste [3]. Therefore, a kitchen waste sorting scheme based on robotic grasping can not only improve the disposal efficiency of traditional methods but also increase the utilization rate of recyclable waste.

Currently, the use of robots to grasp solid waste mainly focuses on conveyor belt scenarios. In this scenario, the waste is piled up on the conveyor belt and detected by the robot's visual servo system, and then, it is grasped by the end-effector to a specified location [4]. Recent studies mainly focus on using RGB-D images and DL-based methods to generate the robot's grasping pose [4], [5], [6]. This method has strong robustness and can recognize the objects to be grasped even if they have slight deformation or dirt. On the other hand, a parallel robot is widely used to pick and place target objects on conveyor belts due to its characteristics of lightweight and fast motion [6]. Therefore, fast grasping of waste on conveyor belts can generally be defined as a 2-D planar grasping task for robots.

Recent research on 2-D planar grasping planning mainly focuses on discriminative method and generative method. Generally, a generative method has fewer steps than a discriminative method [7], [8]. However, to the best of our knowledge, existing generative methods cannot be applied to grasping tasks on conveyor belt. Since the generative grasping planning method is mainly proposed for single-object grasping task, it tends to cause incorrect or excessive grasps when facing multiobject grasping. On the other hand, the efficiency and performance of the grasping model often exhibit a tradeoff [8], [9]. Balancing these two factors plays a pivotal role in the practical deployment and application of the grasping model.

In this article, we aim to develop an efficient generative grasping model for the kitchen waste sorting scenario. To address the challenge of applying generative models to such a scenario, we build a new kitchen waste grasping dataset (KWG2024) and propose novel labeling methods and evaluation metrics to handle the issues of excessive and incorrect grasps. Besides, to balance the efficiency and performance

Received 23 September 2024; revised 1 March 2025; accepted 16 March 2025. Date of publication 31 March 2025; date of current version 10 April 2025. This work was supported in part by the Project of the National Key Research and Development Program of China under Grant 2021YFC1910402, in part by the National Natural Science Foundation of China under Grant 62073129 and Grant U21A20490, in part by Hunan Provincial Natural Science Foundation of China under Grant 2022JJ10020, and in part by the Scientific Research Project of Hunan Education Department under Grant 21B0330. The Associate Editor coordinating the review process was Dr. Yongbo Li. (Corresponding author: Qiaokang Liang.)

Songyun Deng, Ruiyuan Pei, Li Zhou, Hai Qin, and Wei Sun are with the College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: songyun@hnu.edu.cn).

Qiaokang Liang is with the National Engineering Laboratory for Robot Vision Perception and Control, College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: qiaokang@hnu.edu.cn).

Digital Object Identifier 10.1109/TIM.2025.3555698

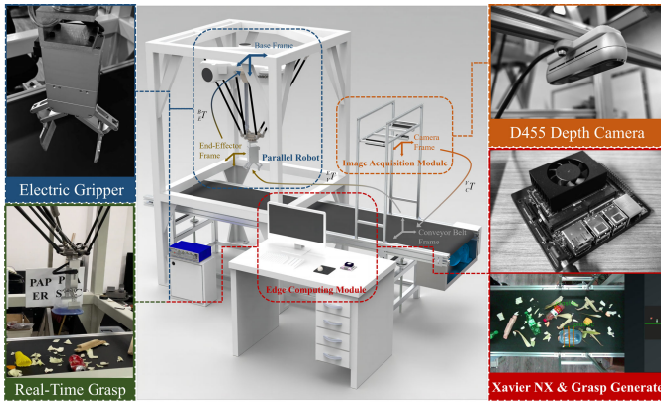


Fig. 1. Overview of the kitchen waste sorting system using our proposed method.

of the network, we propose an efficient grasping model, AMI-LiteNet, which can be deployed on embedded devices with a high inference speed of 111 frames per second (FPS). Moreover, we incorporate the capability of classifying the categories of grasped objects into AMI-LiteNet with a slight computational cost. The main contributions of this article are summarized as follows.

- 1) This work proposes an efficient real-time generative grasping model for kitchen waste sorting. The model exhibits computational complexity with a mere 2.79 GFLOPs and parameters of 1.57 M, enabling it to attain an 87.96% grasping detection accuracy in the intricate task of kitchen waste sorting. Moreover, the ability to classify the grasped objects is added to the model in the form of a classification header. In addition, this model is further deployed on Jetson Xavier NX to indicate its potential for edge computing devices.
- 2) In this work, we present a labeling and training-based method to address the issues of excessive and incorrect grasps arising from generative networks in the grasping task on conveyor belt. Based on this, we constructed the first 2-D grasping planning dataset for kitchen waste sorting. This dataset contains 3116 RGB-D images, ten categories of objects, and over 18 000 human annotations.
- 3) Utilizing our proposed dataset and the corresponding grasping planning method, we implement a conveyor-oriented embedded vision servo platform on a high-speed parallel robot, as shown in Fig. 1. With the efficient inference capability of the vision servo, the robotic system is able to reliably grasp at least 25 targets in a complex environment within 1 min. To the best of our knowledge, this is the first application of a single generative grasping detection model to the kitchen waste sorting task on a conveyor belt.

II. RELATED WORKS

A. Grasp Dataset Based on Oriented Rectangles

Representing a 2-D planar grasp with an oriented rectangle is a simple and easy-to-annotate method. Jiang et al. [10] first proposed the method and used it to construct the Cornell

dataset, the most widely used 2-D grasp dataset [11]. The annotation expert can represent a grasping action in this dataset by confirming the coordinates of three vertices of an oriented rectangle in an image. Based on this representation, they evaluated grasping accuracy using both the intersection of union and angular deviation. However, the limited number of RGB-D images (only 885) in this dataset restricts its applicability to a wider range of studies. To address the quantity problem of the Cornell dataset, Depierre et al. [12] proposed the Jacquard dataset. Unlike the Cornell dataset, it identifies correct grasps based on virtual experiments in a simulation scenario. This dataset contains 54 000 images and enables a more diverse representation of grasps. On the contrary, the annotation does not provide information about the grasp priority because the grasp position can be arbitrary on an object. This leads to the challenge of selecting the optimal grasp on one object with multiple grasping annotations.

B. Grasp Method for Conveyor Belts

Prior to the adoption of deep learning techniques for conveyor-based grasping tasks, visual servoing predominantly relied on template matching or edge detection for generating grasps [4], [13]. The task of grasping on a conveyor belt was generally improved based on object detection after the learning-based approach was proposed. Zhihong et al. [14] developed an initial waste sorting system utilizing RPN and VGG networks, tailored for the conveyor belt environment. Besides, Sun et al. [15] introduced a computer vision approach based on CG-YOLO for identifying the precise location and gripping area of coal and gangue. Nevertheless, these two methods, along with other traditional object detection methods, fall short in accurately discerning the angular orientation of the gripper jaws during the grasping process due to their inability to produce oriented rectangles. To mitigate this problem, the proposed solution is to deploy a semantic segmentation network to ascertain the object's geometric center [6]. This will enable the robot to focus on suctioning the object solely at the central point, thus obviating the need for antipodal grasping. An alternative approach involves utilizing target detection to concurrently predict the object's bounding box and oriented rectangles, with the latter being refined by the bounding box to eliminate any superfluous elements [16]. A closely related method is to accomplish the multiobject grasping task by integrating an object detection network with a generative grasping network [17]. However, these two methods have not yet been applied to the conveyor belt task and failed to yield end-to-end detection outcomes.

C. Lightweight Generative Grasp Point method

Generative grasp method is an end-to-end method, which can directly predict the heat map of the grasp. However, to ensure precise pixel-level predictions of the grasp information, there is often a tradeoff between network performance and inference speed. In order to solve this problem, Morrison et al. [18] proposed the GG-CNN, which can give pixel-level predictions in a simple structure. Moreover, Kumra et al. [19] used residual modules to improve the performance of GG-CNN. The GR-ConvNet series of networks

attain a prediction accuracy exceeding 97% on the Cornell dataset while sustaining real-time inference capabilities on a high-performance computing platform. To enhance the network's efficiency, generative networks were incorporated with components such as multiscale structures, attention mechanisms, and channel fusion mechanisms, enabling reduced inference time while maintaining an elevated prediction accuracy [8], [20]. Furthermore, Fu and Dang [9] used depth-wise convolution and improved the model's postprocessing module to realize real-time grasping planning on an edge computing platform. However, none of these lightweight networks can be applied to the multiobject grasping task. Besides, due to the scarcity of publicly available datasets, there is still a large room for improvement in the performance of these networks.

III. METHODS

A. Overview of Grasping Representation

The representation of the 5-D oriented rectangle [18] is commonly used in generative-based grasping methods. To accomplish the task of grasping objects on a conveyor belt, we need to refine this representation and add an extra dimension to indicate the category of the object being grasped. In the robot base frame, a grasp pose can be defined as

$$\mathbf{g}_B = (q_B, \mathbf{P}_B, \theta_B, \omega_B, c_B) \quad (1)$$

where q_B represents the scalar quality measure, which can indicate the chances of grasp success. The grasp location can be selected with the largest value of q_B as the optimal one within the local position range. The gripper's center position can be indicated as $\mathbf{P}_B = (x_B, y_B, z_B)$ in the robot base frame. θ_B and ω_B represent the gripper's rotation around the z -axis and the required gripper width, respectively. Moreover, c_B indicates the category of the object associated with this grasp.

Since our grasp is a 2-D planar grasp, a grasp can be generated based on an N -channel image $I \in \mathbb{R}^{N \times H \times W}$. In the pixel frame, a grasp can be expressed as

$$\mathbf{g}_I = (f(q_I), \mathbf{P}_I, \theta_I, f(\omega_I), c_I) \quad (2)$$

where f denotes the filter, q_I denotes the scalar quality measure in an image, $\mathbf{P}_I = (x_I, y_I)$ denotes the gripper's center position, θ_I is the grasp angle of rotation, and ω_I is the opening width of the gripper. Besides, for each position $\mathbf{P}_I$, the channel number where the maximum value occurs among n channels is defined as the category number. Specifically, each grasp class, which indicates the category of the object being grasped, can be determined by

$$c_I = \arg \max_{i \in n} (c_I^i) \quad (3)$$

where c_I^i represents the value of a pixel with coordinate $\mathbf{P}_I$ on channel i . In addition, the angle of grasp is coded into two components of a unit vector, $\sin(2\theta_I)$ and $\cos(2\theta_I)$, to avoid any discontinuities in data. The final grasp angle can be calculated by $\theta_I = \arctan(\sin(2\theta_I)/\cos(2\theta_I))/2$.

As shown in Fig. 1, the transformation from the pixel frame to the robot base frame involves a series of conversions, which can be expressed as

$$\mathbf{g}_B = {}^B_E T_V T_C^T T_I^C T(\mathbf{g}_I) + \Delta \mathbf{L} \quad (4)$$

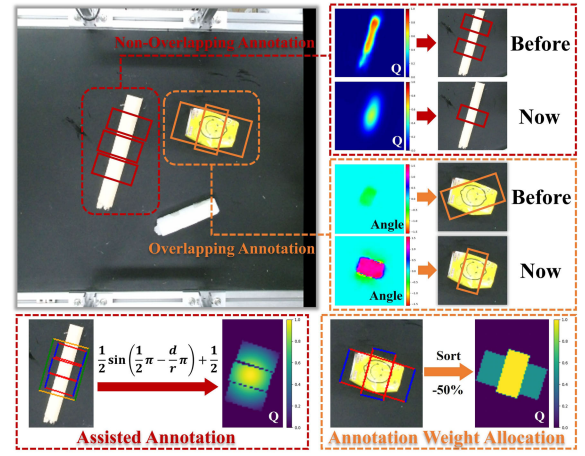


Fig. 2. Schematics depicting the two types of errors resulting from annotation and the predictive outcome following annotation improvement.

where ${}^C_I T$ is the transformation from image frame to camera frame and ${}^V_C T$ is the transformation from camera frame to conveyor belt frame. As objects placed on the conveyor belt shift positions over time, it is necessary to adjust the grasp's pose with a time-varying variable $\Delta \mathbf{L}$ to obtain the grasp pose at each moment, where $\Delta \mathbf{L}$ of the object on the conveyor belt can be acquired through an encoder mounted on the belt. Besides, ${}^E_V T$ is the transformation from conveyor frame to end-effector frame, and ${}^B_E T$ indicates the transformation from end-effector frame to robot base frame. These transformations can be obtained by using the hand-eye calibration algorithm and the RGB-D camera calibration algorithm.

B. Annotation Method

Existing generative 2-D planar grasp methods rely on annotations from the Cornell dataset for grasp planning, using 5-D grasp information provided by feature maps (affordance maps). The advantage of this method is that during annotation, the user only needs to manually select a directed rectangle to define a grasp. During model training, this directed rectangle is then transformed into an affordance map, which can be interpreted by the generative model. Therefore, the basic task of existing generative models is to find the optimal grasping point from an affordance map. Even when dealing with tasks involving multiple grasps, existing methods achieve multi-grasping through the repeated confirmation of the real-world environment [21], that is, after one grasp is completed, a new image is captured to generate the next grasping plan. However, in our task, a robotic vision servo system on a conveyor deduces object poses from a single image, necessitating a generative model to plan grasps for all objects simultaneously. When multiple grasping annotations exist for an object, these generative methods cannot ascertain the affiliation between the grasping annotations and the target object during training. This results in the trained model generating excessive grasps on a single object and leads to an inability to accommodate some grasping angles. We address this by improving the annotation process, categorizing error-prone annotation methods into two types, and suggesting improvements.

As shown in Fig. 2, the first type that can lead to excessive grasping is referred to as “nonoverlapping annotation.” It signifies the presence of multiple nonoverlapping grasping annotations on an object, which results in a generative model that produces multiple grasp plans for the object, leading to an excessive number of grasps. Since the final grasp plans are determined by locating the local maxima of the grasp quality map, this algorithm cannot express whether nonoverlapping grasping annotations belong to the same object. To tackle this problem, we suggest using assisted annotations to direct the model to the nearest grasping annotation to the object’s center. Assisted annotation refers to the construction of an oriented rectangle designed to encompass all rectangles within an object. The central position of this assisted rectangle should be set to align as closely as possible with the center of the rectangle annotation, which is nearest the object’s geometric center. Moreover, in order to employ the assisted rectangle as a mask for setting weights on nonoverlapping annotations of an object, the value of each position within the mask can be calculated using the following formula:

$$D = \frac{1}{2} \sin\left(\frac{1}{2}\pi - \frac{d}{r}\right) + \frac{1}{2} \quad (5)$$

where D is the value of a position within the mask, r denotes the radius of the circumscribed circle around this assisted rectangle, and d represents the Euclidean distance from this position within the mask to its center. The mask’s aim is to give higher weights to positions near the center and lower weights to those further away. Moreover, the use of the sine function serves to increase the coverage area of values approaching 1. This process ensures that the model does not produce a grasp quality map with an ineffective grasping area. On the other hand, this operation is also adaptable to postprocessing using filters with larger Gaussian kernels, which can further reduce the occurrence of excessive grasp. The final annotated quality feature map can be provided by the Hadamard product of the initial feature map and the corresponding mask.

Another type that may lead to incorrect grasping is referred to as “overlapping annotation.” It refers to the partial area overlap among multiple rectangular annotations of an object. As illustrated in Fig. 2, such annotations result in the confusion of information pertaining to different grasping angles and gripper opening lengths within the same range, thereby causing a generative model to output-oriented grasp rectangles with incorrect lengths and angles. In essence, a single-channel feature map can only represent one type of information at a given location, not multiple types. Therefore, our solution (annotation weight allocation) selects one type of information from multiple options to construct the annotated feature map, which is similar to the method used to resolve the excessive grasp error. In detail, this method calculates the intersection over union (IoU) score between any two annotations within an image. Based on our experience, if the IoU between two annotations exceeds 0.27, it is considered to meet the criteria for “overlapping annotation.” Subsequently, these annotations are sorted in descending order by their width ω_I . For each target object, q_I of all annotations that meet the “overlapping annotation” criteria, except for the annotation with

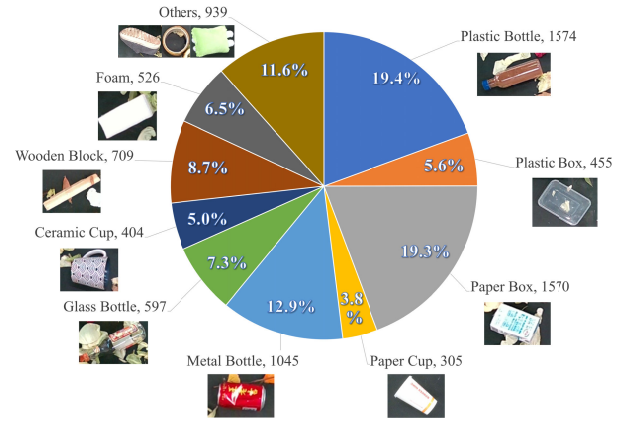


Fig. 3. Sorted distribution of image number per category of KWG2024.

the minimum width, is reduced by 50%. The IoU of two annotations can be calculated as follows:

$$\text{IoU}(G_A, G'_A) = \frac{G_A \cap G'_A}{G_A \cup G'_A} \quad (6)$$

where G_A and G'_A denote any two ground truth rectangle in an image. Besides, the rationale for selecting the annotation with the minimum width as the recommended one is that during robotic grasping, the smaller the required opening width of the gripper, the higher the tolerance for errors in grasping. Hence, by recommending such an annotation and suppressing the expression of other annotations at the same location, we can reduce the occurrence of incorrect grasping.

C. Dataset Construction

Since the complexity and unstructured nature of kitchen waste sorting scenarios, no research within our knowledge scope has yet constructed a dataset for generative models tailored to this context. To address the grasping challenges in such an environment, we develop a robotic grasping pipeline as illustrated in Fig. 1, which simulates the operational setting of an actual waste treatment facility. Our dataset comprises original images captured with Kinect V2 and Realsense D455 cameras. To facilitate model training and robustness testing, we include 2167 RGB-D images taken against a relatively clean background and 949 RGB-D images against a dirty and complex background. The former is captured with the Kinect V2 camera, while the latter is captured with the Realsense D455 camera. Moreover, calibration is performed on both cameras, utilizing the derived parameters to reduce image distortion and to align the RGB images with the depth images.

In this dataset, a total of 8124 instances of waste items intended for grasping are fully represented. Fig. 3 illustrates their specific categories and the exact number of waste items per category. Besides, 238 waste items are partially visible, appearing at the edge regions of the images. To enhance the model’s ability to discern grasping backgrounds, 91 of these images do not contain any waste items intended for grasping. On the other hand, to facilitate better learning of annotation data by the generative model, we stipulate that annotated grasps should closely approximate the object’s

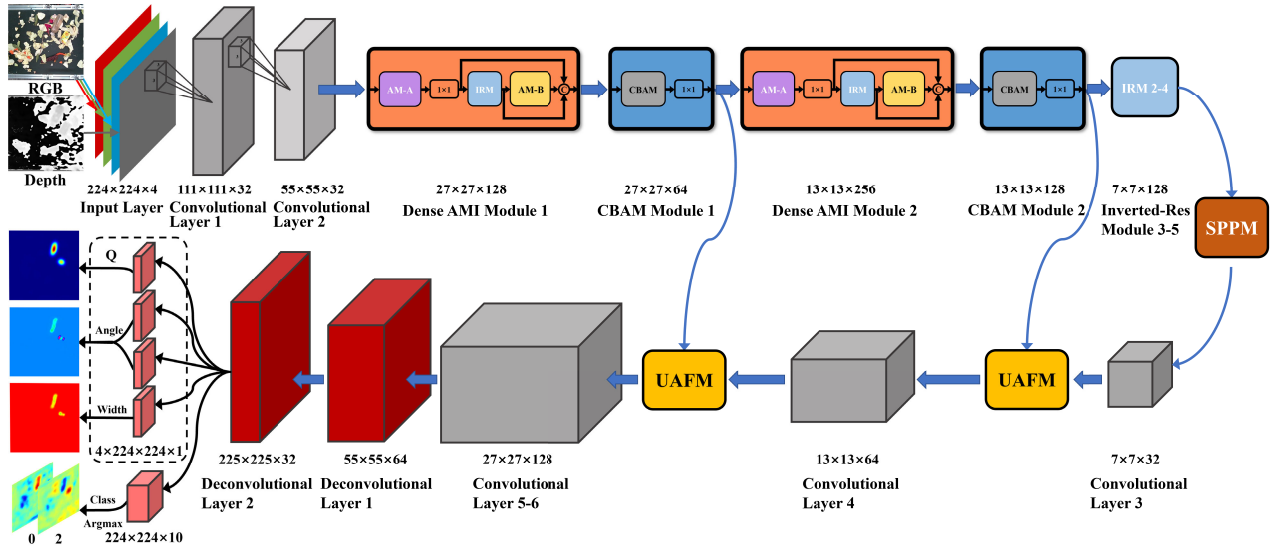


Fig. 4. Proposed architecture: AMI-LiteNet.

centroid or geometric center. Following this guideline and the “assisted annotation” method established in Section III-B, the dataset KWG2024 comprises 2721 assisted annotations and 18059 human-experience annotated grasps based on the annotation standard of the Cornell dataset [10]. In addition, all standard annotations include category labels of the annotated objects. This dataset can be made available at <https://github.com/HNUsong/KWG2024>.

D. Model Architecture

A high-speed, precise algorithm for grasp generation can be easily integrated into various waste sorting facilities. The current generative grasping networks have improved from semantic segmentation networks, with several studies such as SE-ResUNet and GR-ConvNet [19] deriving improvements from classic semantic segmentation networks. Guided by this approach, we aim to construct our network on the foundational architecture of PP-LiteSeg [23] while incorporating advantageous structures from the backbone of AMI-NanoNet [24] to refine our model’s framework. The detailed architecture of our model is depicted in Fig. 4.

In detail, the backbone of the framework is composed of the Dense AMI module, attention mechanisms, and inverted residual module. There are two reasons for not directly adopting the backbone of AMI-NanoNet. First, the channel count per layer in AMI-NanoNet does not uniformly increase with the number of layers. Second, AMI-NanoNet was originally intended for classification tasks, and its computational and parameter volume is too limited for complex tasks like grasp fitting. To overcome this, we recombine the AMI modules proposed in AMI-NanoNet to create a structure that can be repeatedly stacked. The output channel count of the original AMI Module A (AM-A) is four times that of its input, so we add a 1×1 convolution after it to reduce the output channel count. Subsequently, residual connections are established to reuse features from different module outputs. Similarly, after the constructed Dense AMI module, we also use CBAM

and 1×1 convolution to reduce the output channel count. Since 1×1 convolution may result in information loss, CBAM is utilized to mitigate this loss. Finally, the Dense AMI and CBAM modules are used twice in the backbone to correspond to the related structures in the original PP-LiteSeg.

On the other hand, within the decoder section of the structure, we replace the bilinear upsampling of PP-LiteSeg with two layers of deconvolution. This is due to the significant difference between the output of grasp fitting and the input image, compared to semantic segmentation tasks. Therefore, employing two layers of deconvolution instead of direct bilinear upsampling to a size of 224×224 serves to enhance the decoder’s fitting capability. Besides, instead of directly fitting object categories using semantic segmentation, we represent different classes through multiple channels. Formula (3) and Fig. 4 specifically demonstrate this approach. Ultimately, this network has a computational complexity of only 2.79 GFLOPs and contains 1.57 million parameters.

E. Evaluation Criteria

For single-object grasping methods, the rectangle metric is widely used to evaluate the model performance. However, this method, along with existing multigrasp evaluation modes, does not adequately assess the performance of multigrasp models in conveyor belt scenarios. Based on the rectangle metric evaluation benchmark, we propose a new method to evaluate multigrasp models in conveyor belt scenes. For assessing the correctness of a grasp, we employ the evaluation benchmark of the rectangle metric

$$\begin{cases} |\theta_I - \theta_A| < 30^\circ \\ \text{IoU}(G_I, G_A) = \frac{G_I \cap G_A}{G_I \cup G_A} > 0.25 \end{cases} \quad (7)$$

where θ_I is the generated grasp angle, θ_A is the corresponding annotation, G_I indicates the generated grasp rectangle, and G_A denotes the corresponding ground truth rectangle. Based on this formula, we can determine whether an individual grasp

aligns with a standard annotation within the image, indicating whether the grasp can be considered correct. However, in our grasping scenario, a grasp should not only correspond to a specific grasp label but also be determined as nonredundant. Furthermore, if an object has multiple grasping annotations, there should be only one generated grasp associated with that object. To fairly evaluate the generative model's multigrasp performance on a single image, we propose three metrics: image accuracy (IA), grasp accuracy (GA), and classification accuracy (CA), to comprehensively assess the model's grasping and classification capabilities for a single image. Specifically, the calculation method for IA is represented as follows:

$$IA = \frac{M_r}{M}, \quad \text{where} \quad M_r = \sum_{i=1}^M K_i, \quad K_i = \begin{cases} 1, & N'_i = N_i \text{ and } N'_i = n_i \\ 0, & N'_i \neq N_i \text{ or } N'_i \neq n_i \end{cases} \quad (8)$$

where N_i denotes the number of objects that can be successfully grasped in an image and N'_i represents the number of grasps generated within an image. The total number of all tested images is denoted as M . In an image of them, n_i denotes the number of correct grasps generated. The validity of a correct grasp can be verified against all standard annotations in the image by using formula (7). The IA metric, as defined by (8), denotes the proportion of correct images out of the total number of images. An image is considered correct when each object within it is associated with one and only one accurate grasp, without any excessive grasps. Besides, disregarding the robot's grasping efficiency and actual grasp performance, an object can still be considered successfully grasped even if multiple correct grasps are generated for it. Therefore, we require the GA metric to assess the number of objects that may be successfully grasped. It can be denoted as

$$GA = \frac{\sum_{i=1}^M N'_i}{\sum_{i=1}^M N_i}. \quad (9)$$

For all correct grasps, the CA metric is used to determine if these grasps are categorized correctly. It is specifically represented as follows:

$$CA = \frac{\sum_{i=1}^M C_i}{\sum_{i=1}^M n'_i} \quad (10)$$

where C_i indicates the number of correctly categorized grasps among the correct grasps in an image. Overall, the IA metric evaluates the performance of the visual servoing system from the image perspective, reflecting the success rate of grasp detection while considering excessive grasps. The GA metric assesses grasp detection effectiveness from the object perspective, measuring how many objects can be successfully grasped, but it does not account for the impact of excessive grasps. The CA metric is utilized to evaluate the proportion of correctly classified grasps among graspable objects. Thus, based on these three metrics, the performance of a grasp detection model in the KWG2024 dataset can be clearly demonstrated.

It should be noted that in actual robot grasping, the definition of the grasp success rate has a slight difference from

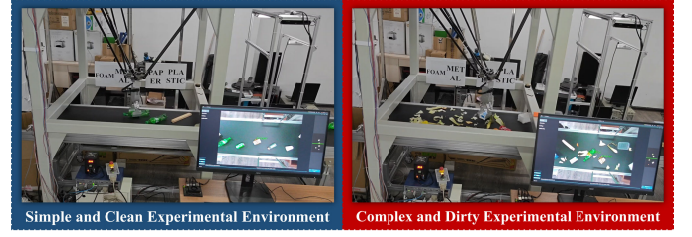


Fig. 5. Robot experimental environment. Left: simple and clean experimental environment. Right: complex and dirty experimental environment. All the tested objects were randomly selected and placed on the conveyor belt at will.

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT

Configuration Item	Server for Training	Jetson Xavier NX
System	Ubuntu 20.04.6 LTS	L4T 35.1.0
CPU	Intel(R) Core(TM) i9-13900K	NVIDIA Carmel ARM@ v8.2
GPU	NVIDIA GeForce RTX 4090 (24G)	NVIDIA Volta™ @ 1100MHz
RAM	128G	8G
CUDA	12.1	11.4
Cudnn	8.9.2	8.4.1
OpenCV	4.9.0	4.7.0
Pytorch	2.2.1	1.12.0
Torchvision	0.18.0	0.13.0
Python	3.11.5	3.8.10

GA. In actual grasping, the grasp success rate is defined as $GA_r = n^*/N^*$, where N^* denotes the total quantity of all objects to be grasped and n^* indicates the total quantity of all the grasped objects [18]. On the other hand, the objects that have been successfully grasped should be placed in the correct positions. Hence, the CA in the actual grasping scenarios is defined as $CA_r = C^*/n^*$, where C^* is the quantity of items that have been classified correctly [4], [25].

F. Loss Function

In this article, the generation of grasp aside from classification is considered a regression problem, and the generation of grasp categories is viewed as a classification problem. Hence, the smooth $L1$ loss is applied as the loss function for the confidence, angle, and width of a grasp, while the cross-entropy loss is used for the grasp classification. Specifically, the loss function can be defined as follows:

$$L = L_{\text{quality}}^s + L_{\text{sin}}^s + L_{\text{cos}}^s + L_{\text{width}}^s + L_{\text{class}}^c, \quad \text{where} \quad L^s(G_I^k, G_A^k) = \begin{cases} 0.5(G_I^k - G_A^k)^2, & \text{if } |G_I^k - G_A^k| < 1 \\ |G_I^k - G_A^k| - 0.5, & \text{otherwise} \end{cases}, \quad \text{and} \quad L^c(C_I, C_A) = - \sum_k \sum_{m=1}^n C(C_I^{km}) \cdot \log(C(C_A^{km})). \quad (11)$$

Specifically, the function C can be calculated as

$$C(a^{km}) = \begin{cases} 1, & \text{if } a^{km} = m \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

where m denotes the category index and k represents the index of the generated grasp and annotated pair combination.

TABLE II
TRAINING ENVIRONMENT ON THE
KWG2024 DATASET

Configuration Item	AMI-LiteNet
Batch Size	8
Optimizer	Adam
Scheduler	OneCycleLR
Max Learning Rate	0.0005
Steps Per Epoch	1
Pct_start	0.16
Diversity Factor	5.5
Final Diversity Factor	2000
Max Epoch	150

IV. EXPERIMENTS AND RESULTS

A. Experimental Setting

Networks' training in this study is conducted on a server platform equipped with a 13th Gen Intel¹ Core² i9-13900K CPU and an NVIDIA GeForce RTX 4090 GPU. These networks are built and trained using PyTorch and after acceleration with TensorRT. After that, they are deployed on the Jetson Xavier NX platform as FP32 data. The specific software and hardware configurations can be found in Table I. Finally, the vision servo system is mounted on a conveyor belt and verifies the grasping success rate of the parallel robot in both simple and complex environments. Fig. 5 shows the real experiment scenario.

Besides, we primarily utilize two datasets. For the comparative experiments in Section IV-B, we conduct experiments using the Cornell dataset. For ablation experiments and the comparative experiments in Sections IV-C and IV-D, we use the KWG2024 dataset. Building upon previous work [10], [19], [20], [21], we perform cross validation on the Cornell dataset with settings, including a batch size of 8, eight num_workers, the Adam optimizer, and an initial learning rate of 0.001. For the KWG2024 dataset, we randomly split it into training and test sets in an 8:2 ratio. For the test set, we augmented each standard annotation with labels for the sequence number of the grasped object, enabling the calculation of IA, GA, and CA evaluation metrics during testing. For the training set, we trained the models using the settings in Table II. Furthermore, for the Cornell dataset, we enhance the input images with random rotation, random cropping, and scaling. For KWG2024, in addition to these enhancement methods, we added random variations in lighting intensity. Drawing on the prior work [18] and our experience, the Gaussian kernel size for postprocessing was set to 5 on KWG2024 and 3 on Cornell.

B. Experiments on Cornell Dataset

Following the configurations described in Section IV-A and formula (7), we evaluated the performance of AMI-LiteNet on the Cornell dataset and compared it with advanced networks from recent years. To ensure training under conditions comparable to other methods, our method was trained for 50 epochs, with each epoch comprising 1000 batches. Moreover, the preprocessing and postprocessing setup for the input

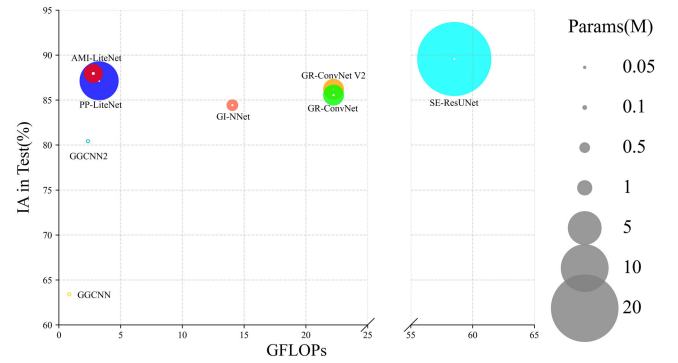


Fig. 6. Performance comparison of AMI-LiteNet and other methods.

data is consistent with the previous work [18]. In order to ensure the reproducibility of the experiment, we conduct the experiment based on the public code environment provided by Kumra et al. [22]. Table III illustrates the efficiency and performance comparison between our method and others. It is noteworthy that the term “test platform” refers to the platform used to test the model, which may not necessarily be the same as the platform used to train the model. In this table, our method can match or exceed the inference speed of other advanced methods that test on high-performance servers, even when only utilizing the edge computing platform. In addition, its precision index is only slightly behind that of GR-ConvNet V2, GI-Nnet, and SKGNet, which are models with high computational complexity and a large number of parameters. It demonstrates the high efficiency of our model in the single-object grasping task.

C. Experiments on KWG2024 Dataset

In this experiment, our aim is to compare the performance of different methods on our dataset. Specifically, the comparison demonstrates different models' performance variations regarding energy efficiency (EE), inference speed, thermal performance, and grasp detection performance under low-power devices (Jetson Xavier NX). The related tests are conducted at an ambient temperature of 18 °C and a relative humidity of 69%, with the Jetson Xavier NX configured to operate under its highest power consumption mode. Since existing generative models lack a classification head, we added one consistent with Fig. 4 to all tested models. Besides, for networks without available code, we consulted their papers and recreated their designs in PyTorch. All input data in testing are uniformly set to a resolution of 224×224 pixels and a modality of RGB-D. To evaluate the inference speed uniformly, we optimized these networks with TensorRT. The tests assessing EE and thermal performance are conducted by configuring the model to run at a speed of 15 FPS on the Jetson device, with the objective of evaluating both the model's EE and the average temperature of CPU or GPU throughout the execution of 10 000 tasks. Specifically, EE is calculated based on Nvidia's definition [28] as the number of tasks executable per milliwatt-hour (mWh). Each task is defined as a model performing a complete inference on a single 224×224 RGB-D image. To minimize the influence of different factors on the experiment, we focused

¹Registered trademark.

²Trademarked.

TABLE III
PERFORMANCE COMPARISON OF MODELS IN CORNELL

Method	Input Size	Modality	GFLOPs	Params	Inference(FPS)	Test Platform	IW(%)	OW(%)
(2020)GGCNN [18]	300 × 300	RGB-D	1.49	72K	588	PC(NVIDIA 3090)	73.9	69
(2020) GGCNN2 [18]	300 × 300	RGB-D	4.24	73K	-	PC	84	82
(2020) GR-ConvNet [19]	224 × 224	RGB-D	22.08	1.9M	263	PC(NVIDIA 3090)	97.7	96.6
(2022) GR-ConvNet V2 [19]	224 × 224	RGB-D	22.07	1.9M	263	PC(NVIDIA 3090)	98.8	97.7
(2022) Tian. et al [8]	224 × 224	RGB-D	-	7.24M	67	PC(NVIDIA 3080)	-	98.9
(2022) SE-ResUNet [20], [26]	224 × 224	RGB-D	58.47	24.04M	40	PC(NVIDIA 1080Ti)	98.2	97.1
(2022) GI-Nnet [9], [27]	224 × 224	RGB-D	13.88	592K	-	PC	98.9	-
(2023) SKGNet [20]	288 × 288	RGB-D	259.14	33.35M	-	PC(NVIDIA TITAN RTX)	99.1	98.4
(2024) QGGNN1 [9]	300 × 300	RGB-D	-	238K	358	PC(NVIDIA 3090)	97.7	98.9
AMI-LiteNet	224 × 224	RGB-D	2.62	1.57M	118	Jetson Xavier NX	98.3	97.1

TABLE IV
PERFORMANCE COMPARISON OF MODELS IN THE KWG2024 DATASET

Method	GFLOPs	Params	Inference(FPS)	EE/(mWh)	Temp(CPU)	Temp(GPU)	Test Platform	IA(%)	GA(%)	CA(%)
GGCNN [18]	0.85	72K	217	33	42.27°C	42.73°C	Jetson Xavier NX	63.40	83.62	37.45
GGCNN2 [18]	2.36	73K	119	24	42.15°C	42.95°C	Jetson Xavier NX	80.42	91.47	45.15
GR-ConvNet [19]	22.24	1.9M	33	10	47.38°C	49.42°C	Jetson Xavier NX	85.55	93.18	93.35
GR-ConvNet V2 [19]	22.24	1.9M	32	10	48.15°C	50.23°C	Jetson Xavier NX	86.20	93.91	93.01
SE-ResUNet [26]	58.51	24.04M	17	5	58.32°C	61.97°C	Jetson Xavier NX	89.57	95.74	91.17
GI-NNet [27]	14.05	594K	40	13	45.99°C	47.47°C	Jetson Xavier NX	84.43	94.09	67.64
PP-LiteSeg [23]	3.26	6.69M	101	22	43.60°C	44.45°C	Jetson Xavier NX	87.14	94.64	98.14
AMI-LiteNet	2.79	1.57M	111	24	43.07°C	43.92°C	Jetson Xavier NX	87.96	95.13	97.76

TABLE V
RESULTS OF ABLATION EXPERIMENTS ON AMI-LITE.NET FOR
DIFFERENT ANNOTATION TECHNIQUES

Assisted Annotation	Annotation Weight Allocation	IA(%)	GA(%)	CA(%)
		83.47	93.48	96.76
✓		86.68	94.03	98.00
	✓	84.43	93.97	95.93
✓	✓	87.96	95.13	97.76

exclusively on measuring the combined EE of the CPU and GPU rather than calculating the EE of the entire device.

Considering the comprehensive evaluation capability of the IA metric, it serves as our primary benchmark. Thus, in this experiment, the selection for the other two metrics (GA and CA) is determined by the epoch that achieves the highest IA value. Table IV shows the multiobject grasping performance of these networks on the KWG2024 dataset and their inference speed, EE, and temperature of CPU/GPU on the NX device. Since the inference speed, EE, and temperature can be influenced by various factors, including code implementation and hardware performance, we primarily focus on computational complexity and parameter count as metrics for evaluating computational performance. The model's inference speed, EE, and temperature metrics are measured to serve as practical validation of these theoretical criteria on actual hardware. From Table IV, it can be observed that models characterized by low computational complexity and fewer parameters typically exhibit higher EE, faster inference speeds, and lower operating temperatures when implemented on hardware.

Furthermore, Fig. 6 presents a performance comparison of these methods. The x -axis represents computational cost, while the y -axis shows the IA value. The area of each circle reflects the model's parameter count. Therefore, in this figure, models closer to the top-left corner exhibit higher performance efficiency, and smaller circle areas indicate fewer

parameters. Evidently, our proposed method outperforms other networks in the multiobject grasping task for the conveyor belt scenario, which has demonstrated exceptional performance in single-object grasping. Moreover, our model's performance is significantly superior to that of networks with fast inference speed, and it holds a greater advantage in both IA and inference speed compared to models with good performance. Overall, our method achieves the optimal balance between inference speed and IA metric.

D. Ablation Study

The ablation study necessitates presenting the impact of our proposed baseline, the number of deconvolution layers, and two novel annotation processing methods on model training. We trained AMI-LiteNet based on the configurations in Table II and the utilization of different annotation handling techniques. The results are illustrated in Table V. It is evident that each improvement measure significantly enhances the model's performance, and using both methods concurrently yields optimal results. On the other hand, based on the method proposed in Section III-D, we need to validate that using deconvolution at the network's end outperforms the original structure in PP-LiteSeg. In addition, the number of deconvolution layers may significantly affect the performance, which should be assessed in the ablation study. Therefore, based on the two annotation improvements mentioned above, we conducted an ablation study on the network's end structure. This experiment is carried out on both AMI-LiteNet and PP-LiteSeg structures, and the results are shown in Table VI, which indicates that a two-layer deconvolution structure yields the best performance in both models. In addition, increasing the number of deconvolution layers significantly raises the computational burden, while omitting deconvolution layers adversely affects grasping performance. Moreover, the results show that too many deconvolution layers lead to a decline in grasping performance. Therefore, using two deconvolution

TABLE VI
RESULTS OF ABLATION EXPERIMENTS ON AMI-LITE NET AND PP-LITESEG FOR DIFFERENT DECONVOLUTION LAYERS

Method	1 Deconvolution Layer	2 Deconvolution Layers	3 Deconvolution Layers	GFLOPs	Params	IA(%)	GA(%)	CA(%)
AMI-LiteNet	✓			2.28	1.74M	87.80	95.31	98.41
AMI-LiteNet		✓		2.62	1.57M	87.96	95.13	97.76
AMI-LiteNet			✓	3.27	1.49M	86.52	94.46	97.49
AMI-LiteNet				1.51	1.22M	84.91	93.42	98.57
PP-LiteSeg	✓			2.92	6.95M	86.68	94.46	98.20
PP-LiteSeg		✓		3.26	6.69M	87.14	94.64	98.14
PP-LiteSeg			✓	3.75	6.62M	87.00	94.95	98.21
PP-LiteSeg				1.97	6.34M	86.20	94.15	98.45

TABLE VII
RESULTS OF GRASPING OBJECTS WITH DIFFERENT PROPERTIES

Properties	GA _r (%)		CA _r (%)	
	Simple Scenario	Complex Scenario	Simple Scenario	Complex Scenario
Partial Transparency	94.12	92.59	100.00	95.78
Non-transparency	88.89	88.06	98.00	93.22
Excessive Reflection	87.50	84.09	95.92	91.89
Normal Reflection	93.42	93.51	98.57	97.22

layers achieves the optimal balance between computational load and grasping performance.

E. Experiments on Robotic Platform

To assess our methods' grasping efficacy in real robotic systems, we set up simple and complex scenarios for experiments depicted in Figs. 1 and 5. In these scenarios, the conveyor belt remains operational (80 mm/s speed), and the robot executes the grasping action once the object to be picked reaches the graspable area. Due to constraints in the actual space and the variety of graspable objects, our experiment does not differentiate between object shapes but focuses on material types. We limited our tests to four categories, treating wooden blocks as paper, which are plastic, paper, metal, and foam. Besides, objects were randomly placed on the conveyor belt by experimenters, and the robot grasped them in the order they arrived. Ultimately, the system achieved a grasping success rate (GA_r) of 90.91% in a simple scenario and 90.08% in a complex scenario. Among the successful grasps, the CA (CA_r) is 97.48% in a simple scenario and 95.41% in a complex scenario.

Moreover, in these grasping scenarios, we also calculated the grasping success rate and CA concerning objects with transparent features or excessive reflection properties. The comparisons of these results can be found in Table VII. In particular, we believe that for this type of task, the impact caused by excessive reflection is greater than that caused by the transparency of objects. We attribute this because objects causing excessive reflection, both in terms of variety and quantity, significantly outnumber transparent objects among the items requiring sorting. In our KWG2024 dataset, only plastic and glass-type graspable objects exhibit the transparent property. Meanwhile, wooden items, paper, and foam may possess excessive reflective characteristics. Furthermore, complex scenes increase the difficulty of classifying and grasping these two types of objects in our system. However, in complex scenarios, our system achieves a grasping accuracy exceeding

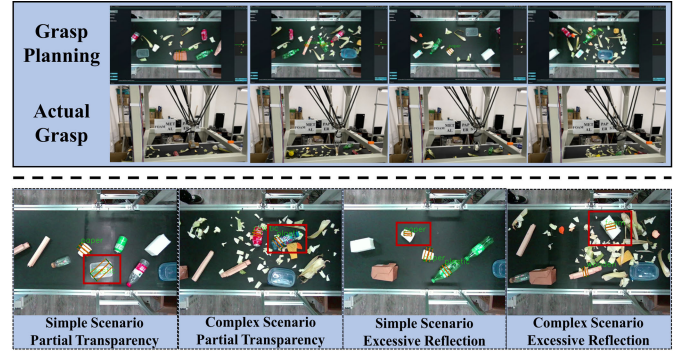


Fig. 7. Experiments of our method in actual scenarios. Grasp planning refers to the grasp postures inferred by our visual servo system. Actual grasp refers to the grasping performed by the robot in the real world. In the figure, the upper part displays our grasping scenarios, while the lower part illustrates grasp failures caused by the properties of partial transparency or excessive reflection.

84% for objects with excessive reflection property and a CA surpassing 91%, despite the increased probability of grasp failure associated with these two types of objects. It can be seen from Fig. 7 that our real-world robot experiments are presented and some failure examples can be found. Overall, our system's performance in complex scenario is slightly lower than in simple scenario. Failures in grasping and classification are primarily due to excessive reflection from objects' surface and rolling of them.

V. CONCLUSION AND DISCUSSION

In this article, we introduce AMI-LiteNet and two annotation handling methods to tackle the challenge of multiobject waste sorting on conveyor belt. We compiled the KWG2024 dataset featuring ten waste categories from varied scenarios due to the shortage of public data. Furthermore, we conducted comparative experiments on the Cornell dataset, as well as ablation and comparative studies on KWG2024. The results indicate that our model is efficient in both single-object and multiobject grasping tasks. Ultimately, our method demonstrates a grasping success rate of over 90% and a CA of over 95% in a robotic system equipped with an embedded vision system, with only a slight performance decrease in simple scenario compared to complex one. The model's swift and precise inference on embedded devices indicates its adaptability to various robotic systems, underscoring the promise of generative methods for conveyor belt applications. Moreover, various tests indicate that it is able to generate high-precision grasping detection results efficiently for our model on low-power devices.

Besides, there are still some issues in this scenario that need to be addressed. For instance, although the proposed annotation method enables the use of generative grasping techniques in this scenario, it still involves a considerable amount of manual work. In future research, we plan to optimize this by combining automatic annotation with manual verification. Specifically, our strategy entails training a model based on the KWG2024 dataset to perform grasping and sorting of unseen objects. To achieve this, we will utilize an end-effector equipped with sensory capabilities that autonomously determines the success or failure of each grasp. This process enables the automatic annotation of images captured during sorting activities. Following this, manual verification and correction steps are implemented to semiautomatically generate new datasets. In addition, the proposed method still leaves room for improvement in terms of grasping success rate in highly complex and dirty environments. We are considering the adoption of a dynamic learning model structure in future studies to further enhance adaptability across different scenarios. Furthermore, we have invested significant effort in tuning hyperparameters. After testing several configurations, we identified the settings shown in Table II as the most effective. However, the large number of hyperparameters increases the experimental workload. We aim to simplify these settings in future research.

REFERENCES

- [1] A. M. Madsen et al., "Review of biological risks associated with the collection of municipal wastes," *Sci. Total Environ.*, vol. 791, Oct. 2021, Art. no. 148287.
- [2] Q. Meng, H. Liu, H. Zhang, S. Xu, E. Lichtfouse, and Y. Yun, "Anaerobic digestion and recycling of kitchen waste: A review," *Environ. Chem. Lett.*, vol. 20, no. 3, pp. 1745–1762, Feb. 2022.
- [3] L. Fang et al., "Long-tailed object detection of kitchen waste with class-instance balanced detector," *Sci. China Technol. Sci.*, vol. 66, no. 8, pp. 2361–2372, Aug. 2023.
- [4] T. Kiyokawa, J. Takamatsu, and S. Koyanaka, "Challenges for future robotic sorters of mixed industrial waste: A survey," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 1023–1040, Jan. 2022.
- [5] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3212–3232, Nov. 2019.
- [6] S. Um, K.-S. Kim, and S. Kim, "Fast suction-grasp-difficulty estimation for high throughput plastic-waste sorting," *J. Mech. Sci. Technol.*, vol. 37, no. 2, pp. 955–964, Feb. 2023.
- [7] H. Cheng, Y. Wang, and M. Q.-H. Meng, "A robot grasping system with single-stage anchor-free deep grasp detector," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–12, 2022.
- [8] H. Tian, K. Song, S. Li, S. Ma, and Y. Yan, "Lightweight pixel-wise generative robot grasping detection based on RGB-D dense fusion," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–12, 2022.
- [9] K. Fu and X. Dang, "Light-weight convolutional neural networks for generative robotic grasping," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6696–6707, Apr. 2024.
- [10] Y. Jiang, S. Moseson, and A. Saxena, "Efficient grasping from RGBD images: Learning using a new rectangle representation," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2011, pp. 3304–3311.
- [11] G. Du, K. Wang, S. Lian, and K. Zhao, "Vision-based robotic grasping from object localization, object pose estimation to grasp estimation for parallel grippers: A review," *Artif. Intell. Rev.*, vol. 54, no. 3, pp. 1677–1734, 2021.
- [12] A. Depierre, E. Dellandrea, and L. Chen, "Jacquard: A large scale dataset for robotic grasp detection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 3511–3516.
- [13] Y. Yang, J. Yang, H. Zeng, and J. Li, "A 3D vision-based conveyor tracking system for pick-and-sort robotic applications," in *Proc. 6th Int. Conf. Robot. Autom. Eng. (ICRAE)*, Nov. 2021, pp. 231–236.
- [14] C. Zhihong, Z. Hebin, W. Yanbo, L. Binyan, and L. Yu, "A vision-based robotic grasping system using deep learning for garbage sorting," in *Proc. 36th Chin. Control Conf. (CCC)*, Jul. 2017, pp. 11223–11226.
- [15] Z. Sun, L. Huang, and R. Jia, "Coal and gangue separating robot system based on computer vision," *Sensors*, vol. 21, no. 4, p. 1349, Feb. 2021.
- [16] Y. Song, L. Gao, X. Li, W. Shen, and K. Peng, "A novel vision-based multi-task robotic grasp detection method for multi-object scenes," *Sci. China Inf. Sci.*, vol. 65, no. 12, Dec. 2022, Art. no. 222104.
- [17] J. Zheng, Y. Zou, J. Xu, and L. Fang, "A method for object recognition and robot grasping detection in multi-object scenes," in *Proc. Int. Conf. Intell. Robot. Appl.*, Jan. 2022, pp. 189–196.
- [18] D. Morrison, P. Corke, and J. Leitner, "Learning robust, real-time, reactive robotic grasping," *Int. J. Robot. Res.*, vol. 39, nos. 2–3, pp. 183–201, Mar. 2020.
- [19] S. Kumra, S. Joshi, and F. Sahin, "GR-ConvNet v2: A real-time multi-grasp detection network for robotic grasping," *Sensors*, vol. 22, no. 16, p. 6208, Aug. 2022.
- [20] S. Yu, D.-H. Zhai, and Y. Xia, "SKGNet: Robotic grasp detection with selective kernel convolution," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 4, pp. 2241–2252, Oct. 2023.
- [21] F.-J. Chu, R. Xu, and P. A. Vela, "Real-world multiobject, multigrasp detection," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 3355–3362, Oct. 2018.
- [22] S. Kumra, S. Joshi, and F. Sahin, "Antipodal robotic grasping using generative residual convolutional neural network," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 9626–9633.
- [23] J. Peng et al., "PP-LiteSeg: A superior real-time semantic segmentation model," 2022, *arXiv:2204.02681*.
- [24] S. Deng, W. Wu, K. Zou, H. Qin, L. Cheng, and Q. Liang, "Efficient real-time recognition model of plant diseases for low-power consumption platform," *IEEE Trans. Artif. Intell.*, vol. 5, no. 5, pp. 2040–2054, May 2024.
- [25] M. Koskinopoulou, F. Raptopoulos, G. Papadopoulos, N. Mavrakis, and M. Maniadakis, "Robotic waste sorting technology: Toward a vision-based categorization system for the industrial robotic separation of recyclable waste," *IEEE Robot. Autom. Mag.*, vol. 28, no. 2, pp. 50–60, Jun. 2021.
- [26] S. Yu, D.-H. Zhai, Y. Xia, H. Wu, and J. Liao, "SE-ResUNet: A novel robotic grasp detection method," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 5238–5245, Apr. 2022.
- [27] P. Shukla, N. Pramanik, D. Mehta, and G. C. Nandi, "Generative model based robotic grasp pose prediction with limited dataset," *Appl. Intell.*, vol. 52, no. 9, pp. 9952–9966, Jul. 2022.
- [28] N. Corp. (2023). *Energy Efficiency*. [Online]. Available: <https://www.nvidia.com/en-us/glossary/energy-efficiency/>